E

Troubleshooting & debugging agents

> **Note**
>
> *Symptom-first reference. Most problems fall into one of the buckets below. Read the "Likely cause" first: it usually tells you whether the fix is going to be thirty seconds or ten minutes.*

Every fix in this appendix ends in the same way: paste what you saw, ask Claude what broke. The numbered steps are scaffolding for when paste-and-ask isn't enough. That move, *paste the error back*, is the rule Maya uses in *Chapter 1* and the rule that runs underneath every entry below.

A note before you dive in: there are several ways to install Claude Code (native installer, Homebrew on Mac, WinGet on Windows, npm). The fixes below cover the most common paths. If a fix references a tool you didn't use to install (`npm`, when you used the native installer), skip that step and try the next.

Symptom index

If you saw...	Go to entry
Install · auth · runtime	
`command not found` after install	1
`npm command not found` (Mac / Windows)	2, 3
`Permission denied` / `EACCES` on Mac	4
Auth fails or browser tab hangs at login	5
Claude hangs after a prompt	6
Output quality	

If you saw...	Go to entry
Confidently wrong / hallucinated output	7
Refuses something reasonable	8
Forgets context mid-conversation	9
Output too long or too short	10
Tone is wrong	11
Things you generated	
HTML file looks broken	12
Script throws an error	13
Limits and platforms	
Rate-limit / "too many requests"	14
Mac vs. Windows behavior diverges	15
MCP set up but no access	16
Agent-era failures	
A hook hangs the session	17
MCP returns malformed or empty data	18
Subagent loops	19
Scheduled job didn't fire	20
Permission denied unexpectedly	21
Two agents fighting over the same file	22
A skill stopped being picked up	23
Sudden cost spike	24

Section 1: Foundation troubleshooting

1. `command not found` **after install.**

Likely cause: The install finished, but your terminal doesn't know where to find the new command yet, usually because your shell was opened before the install updated your PATH (the list of folders your terminal searches for commands).

Fix:

1. Quit the terminal completely (*Cmd* + *Q* on Mac, close all windows on Windows).
2. Open a new terminal.
3. Type `claude --version`. You should see a version number.
4. If it still says `"command not found"` and you used the native installer, run `~/.local/bin/claude --version`. If that works, add `~/.local/bin` to your PATH: `echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc`, then `source ~/.zshrc`.
5. If you used Homebrew: `brew reinstall claude-code`.
6. If you used npm: `npm list -g | grep claude-code` to confirm it's installed; `npm config get prefix` to find where npm puts commands; ensure that path is in your PATH.

2. On Mac, `npm` says "command not found".

Likely cause: Node.js isn't installed yet, or the terminal can't find it.

Fix:

1. Go to `nodejs.org`.
2. Download the LTS (Long-Term Support: the stable, recommended) version.
3. Run the `.pkg` installer.
4. Quit and reopen your terminal.
5. `npm --version` should now return a version.
6. If you use Homebrew, `brew install node` is an alternative: pick one method, not both.

3. On Windows, `npm` is not recognized.

Likely cause: Node.js isn't installed, or it was installed without the "Add to PATH" option.

Fix:

1. Go to `nodejs.org`, download the Windows LTS `.msi`.
2. Run the installer. Make sure **Add to PATH** is checked.
3. Close every PowerShell or Command Prompt window.
4. Open a brand new PowerShell.
5. `npm --version` should return a version. If not, restart the machine: Windows sometimes needs a full restart for PATH changes.

4. On Mac, `Permission denied` or `EACCES`.

Likely cause: npm trying to write into a system folder your account can't touch. Only happens with npm-based installs in the wrong location.

Fix:

1. Don't use `sudo` to fix it: works once, causes worse problems later.

2. Easiest fix: switch to the native installer: `curl -fsSL https://claude.ai/install.sh | bash`.
3. To stay on npm, set up a user-owned prefix: `mkdir ~/.npm-global` then `npm config set prefix '~/.npm-global'`.
4. Add it to PATH: `echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.zshrc` (or `~/.bash_profile` for bash).
5. `source ~/.zshrc`. Then reinstall: `npm install -g @anthropic-ai/claude-code.`

5. Authentication fails or the browser tab hangs at login.

Likely cause: Browser blocked the redirect, you're signed into a different Anthropic account, or there's a transient outage.

Fix:

1. *Ctrl* + *C* to cancel, type `claude` to start over.
2. Sign into `claude.ai` with the email tied to your paid plan.
3. If you have multiple accounts, sign out of all of them in the browser first.
4. If the browser tab opens but never returns you to the terminal, look at the terminal: it usually displays a code or asks you to paste a URL.
5. Try a different browser if the login button does nothing.
6. If nothing works, check `status.anthropic.com` for an outage.

6. Claude hangs after a prompt.

Likely cause: Long prompts can take 30+ seconds; or the connection dropped silently.

Fix:

1. Wait a full minute: most "hangs" are slow responses.
2. *Ctrl* + *C* once to cancel.
3. Try a short test message: "say hello." If that responds, the previous prompt was just slow.
4. If short messages also hang, exit Claude Code and restart.
5. Check your internet connection by loading a website in a browser.

Section 2: Output quality

1. Confidently wrong / hallucinated output.

Likely cause: The model is generating plausible-sounding output instead of looking things up.

Fix:

1. Never trust names, numbers, dates, quotes, or URLs without checking.
2. Paste the real source material and tell Claude to answer based only on what you gave it. *"Here is the article. Answer only from this: if it isn't in here, say so."*

3. Add to your prompt: *"If you don't know, say so, don't guess."*
4. Cross-check anything load-bearing with a search engine or expert.

2. Refuses something reasonable.

Likely cause: Your prompt accidentally tripped a safety filter, or earlier context made it cautious.

Fix:

1. Reread the prompt: could any word be misread?\
2. Rephrase in simpler words. Explain the goal.
3. Add context: *"I'm working on [TASK] for [LEGITIMATE REASON]."*
4. If it still refuses, start a new conversation.

3. Forgets context mid-conversation.

Likely cause: Conversation has grown past the context window. Earlier parts are dropping out to make room.

Fix:

1. Summarize the important context in one message and continue. *"Quick recap: I'm writing a blog post about X for audience Y."*
2. For load-bearing context, save it to a file and paste back when needed.
3. Start a new conversation if it's getting unwieldy. The structural fix is your `CLAUDE.md` from *Chapter 5*: for anything you keep re-pasting, file it there once.

4. Output too long or too short.

Likely cause: You didn't specify length, so the model guessed.

Fix:

1. Be exact: *"Under 150 words." "Three paragraphs." "One sentence."*
2. If too long: *"Cut this in half without losing the important bits."*
3. If too short: *"Expand, add a concrete example and more detail on [SECTION]."*
4. For repeatable tasks, bake length into your template.

5. Tone is wrong.

Likely cause: Vague tone words ("professional," "casual") get interpreted differently than you meant.

Fix:

1. Show, don't tell. Paste an example of the tone and say *"Match this voice."*
2. Name what you don't want: *"No exclamation points. No 'leverage' or 'passionate.'"*
3. Use a persona: *"Write like a friendly accountant explaining taxes to a neighbor."*
4. If close but not right, name what's off: *"Less formal. Drop the marketing words."*

Section 3: Things you generated

1. HTML file looks broken in a browser.

Likely cause: File incomplete, CSS not where you expected, or the wrong file is open.

Fix:

1. Use **Open With | Your Browser** instead of double-clicking.
2. View page source to confirm the HTML saved.
3. Open the browser's developer tools (*F12* / *Cmd + Option + I*) and check **Console**.
4. Tell Claude exactly what's wrong: *"The text is invisible"* or *"Layout is stacked when it should be side by side."*
5. Ask: *"Give me the complete HTML file from scratch with all the CSS in a* `<style>` *tag, so I can paste the whole thing."*

2. Script throws an error.

Likely cause: Missing dependency, typo, or wrong assumption about your setup.

Fix:

1. Copy the entire error: every line.
2. Paste it back: *"This script threw this error. What's wrong and what's the smallest fix?"* (This is *Appendix A*'s universal paste-the-error job.)
3. Tell Claude your setup: OS, where the file is, what you typed.
4. Don't accept a new version blindly: ask what caused it so you catch it next time.
5. *"Module not found"* usually means `pip install [name]` or `npm install [name]`.

Section 4: Limits and platforms

1. Rate-limit / "too many requests."

Likely cause: Account tier has a per-hour or per-day cap. You've hit it.

Fix:

1. Wait. The error usually tells you how long.
2. Use the pause to triage what's actually worth asking next.
3. If you hit limits often, your plan is too small: upgrade.
4. Batch remaining questions instead of one-at-a-time.
5. For full context on limits and observability, *Appendix B*.

2. Mac vs. Windows behavior diverges.

Likely cause: Different shells, different path separators (\ vs /), different defaults.

Fix:

1. Tell Claude your OS at the start: *"I'm on Windows 11 using PowerShell."*
2. When asking for scripts, specify the shell: *"Give me a PowerShell command"* or *"a bash command for WSL"* (Windows Subsystem for Linux, runs a Linux environment inside Windows).

3. Use the right path format (`C:\Users\You\file.txt` on Windows, `/Users/you/file.txt` on Mac).
4. For a Mac-like experience on Windows, install WSL.

3. MCP set up but no access.

Likely cause: The connector needs authentication, or it's connected to the wrong account.

Fix:

1. In a Claude Code session, run `/mcp`.
2. Find the connector and check its status. If it says it needs authentication, select it and choose **Authenticate**: a browser opens; sign in and approve.
3. If it's connected but to the wrong account, reconnect it with the right one.
4. Re-run your prompt; the connector's tools should now be available.
5. If the connector isn't listed at all, add it again the way you did in *Chapter 7*: a directory connector from `claude.ai/directory`, or a by-URL server with `claude mcp add`.

Section 5: Agent-era failures

These are the eight that didn't exist in the first edition's universe. Each is paired with one short confessional aside in italics: the time it bit me.

1. A hook hangs the session.

Likely cause: Hook command makes a network call with no timeout, or has an infinite loop.

Fix:

1. Test the hook command directly outside the hook context. If it hangs there too, the problem is in the command, not the hook plumbing.
2. Add a timeout. For a `curl`, that's `curl --max-time 5`. For other commands, ask Claude to add a timeout appropriate to that tool.
3. Capture `stderr` (the error-output stream a command writes when something goes wrong) to a known log path (*Appendix B*, *Section 3*).
4. Paste the hook config and the hang symptom: *"My SessionStart hook hangs. Here's the JSON: [PASTE]. Here's the command's stderr after I tested it directly: [PASTE]. Diagnose."*

I once spent forty minutes diagnosing a "hook hang" that turned out to be a `curl` *with no timeout against a dead endpoint. Test the command outside the hook first.*

2. An MCP returns malformed or empty data.

Likely cause: Upstream API changed, auth expired silently, or the MCP server crashed and is returning fallback nulls.

Fix:

1. Smoke-test the MCP from a scratch session: ask Claude to call the simplest tool with the simplest input.

2. Paste the smoke-test output back: *"the [MCP] returned [PASTE] when I expected [WHAT]. Diagnose: auth, schema drift, or upstream change?"*
3. Check the MCP's log at `~/.claude/mcp-logs/[server-name].log` (*Appendix B*).
4. If auth: re-run the MCP's auth flow (`claude mcp` commands).

I had a Notion MCP return empty arrays for two days before I noticed. Auth had expired silently. Now my watchdog pings me when an MCP that returned data yesterday returns empty today.

3. A subagent loops.

Likely cause: The agent prompt allows recursion without a base case, or the trigger event re-fires on the agent's own output.

Fix:

1. Kill the session immediately. *Ctrl* + *C* twice if needed.
2. Read the agent file and the last 3 invocations' transcripts.
3. Paste both back: *"This agent looped 12 times. Here's the agent file: [PASTE]. Here are the last 3 invocations: [PASTE]. Find the loop trigger and propose a stopping condition."*
4. Add the stopping condition. Re-test against a known input that triggered the loop.

Iris invoked herself when a Stop hook I'd written posted to the same channel that triggered her. The output of one was the input of the next. The fix was three lines.

4. A scheduled job didn't fire.

Likely cause: One of four things, paused, machine asleep at fire time, prior exit code disabled it, or cron expression (the schedule rule for cron, Unix's job scheduler) interpreted differently than you expected.

Fix:

1. Ask Claude: *"List scheduled jobs with last fire time and last exit code. Job [ID] should have fired at 6:30. Diagnose which of: paused, asleep-at-fire-time, prior-exit-disabled-it, or cron-misread."*
2. Diagnose from the table.
3. For "asleep at fire time," check your machine's wake schedule.
4. For "prior exit disabled it," check the job's run-and-verify and the agent's last log.

The Tuesday my morning brief went missing, the cause was that I'd put the laptop to sleep at 5 AM and the job was set to run at 6:30 with no wake-on-schedule. The fix was a system-preferences toggle, not a code change.

5. Permission denied unexpectedly.

Likely cause: A rule in `.claude/settings.json` (or `settings.local.json`) blocks the action. Often a rule that matched something you didn't intend.

Fix:

1. *"Read* `.claude/settings.json` *and* `.claude/settings.local.json`*. Tell me which rule blocked the action [DESCRIBE]."*
2. Claude reads both, identifies the matching rule.

3. Propose the smallest change to allow this case without opening the door wider. (*Appendix F* has the annotated permissions block to reference.)
4. Cross-reference *Chapter 10* if you want the model behind permissions.

The first time this hit me, I'd added a deny rule for `rm -rf` *against my data folder and forgotten that an unrelated cleanup script I wrote a year before used* `rm -rf` *for tempfiles in a different folder. The rule pattern was too broad. I scoped it.*

6. Two agents fighting over the same file.

Likely cause: No file-lock convention; both agents write to the same artifact path. The race shows up as a half-written file or an overwrite.

Fix:

1. Make one agent the writer, the other the reader. Read the agent prompts and decide which is canonical.
2. *"Restructure these two agents so [AGENT-A] writes [FILE] and [AGENT-B] reads it. Update both agent files."*
3. If both genuinely need to write, introduce a queue file or sub-paths per agent.

Reuben and Iris both tried to write to `~/work/journal/2026-04.md` *at end of week. The journal had only the last writer's content. Now Iris writes to* `journal/iris-2026-04.md` *and Reuben writes to* `journal/reuben-2026-04.md` *and a separate digest agent merges them.*

7. A skill stopped being picked up.

Likely cause: Skill file moved, name mismatch, or the trigger description got too vague during a refactor.

Fix:

1. *"Why isn't* `.claude/skills/[NAME].md` *being invoked when I [TRIGGER]? Read the skill, identify the gap, propose the prompt fix."*
2. Claude reads the skill file and the situation that should trigger it.
3. The fix is usually a sharper trigger description in the skill metadata.

I renamed a skill from "debugging" to "debug" thinking shorter was better. Six skills downstream stopped firing because their `CLAUDE.md` *references hadn't updated. The fix was a five-minute search-and-replace.*

8. Sudden cost spike.

Likely cause: A hook re-loading a large file every session, an MCP returning enormous payloads, or a subagent multiplier that wasn't capped.

Fix:

1. Run job 25 from *Appendix A*: the cost-trend check.
2. Cross-reference *Appendix B*'s "what blows up your bill" sidebar.
3. The diagnosis usually points at one of three things: a `CLAUDE.md` that grew, an MCP returning fuller payloads than it used to, or a subagent that started recursing.
4. Apply the fix at the source. The watchdog (*Chapter 20*) catches the next occurrence; the cost-trend job catches the *next* next one.

I left a SessionStart hook that re-fetched a Gmail thread for context every time I opened a new tab. By month-end the bill caught my attention. Now I cache the fetch in the journal and the hook reads the cached file: same context, almost no cost.

If your symptom isn't here, the universal fallback is the rule the whole appendix is built on: paste what you saw, ask what broke. *Appendix A*'s job 19 is the one-liner to keep in your paste-buffer. Most days that's the only entry in this appendix you actually need.